An experimentation with SCHED_EXT Linux scheduling class

Matteo Di Noia
Matricola 979620

Relatore: Enrico Bini
University of Turin

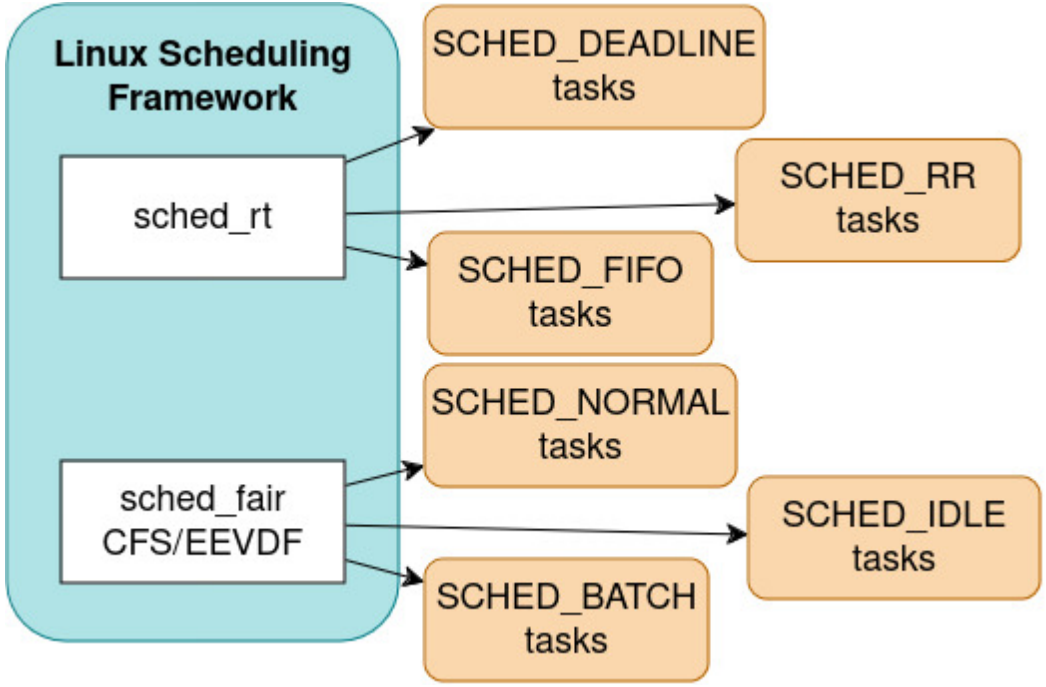
Correlatore: Michele Castrovilli
University of Turin

Scheduler

A scheduler is an essential kernel component of any OS which does:

1. **Task selection**: chooses the task to be run next;
2. **Time Slice**: chooses how long the next task is allowed to run;
3. **Core selection**: chooses which CPU core should be used.

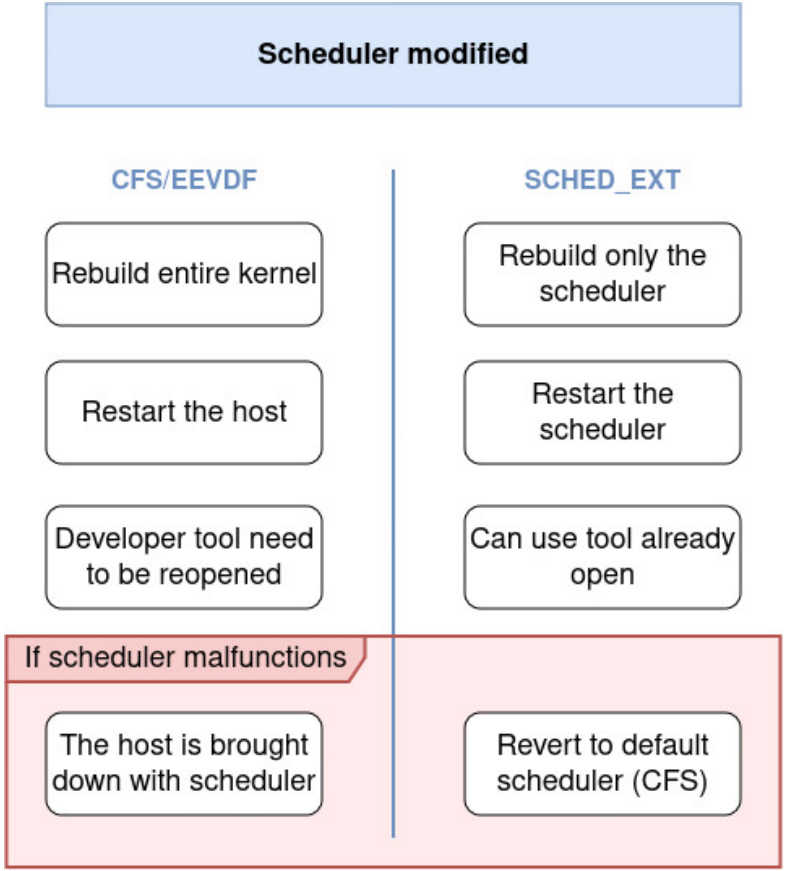
In the Linux kernel, there are various **scheduling classes** that are on different levels of priority.



SCHED_EXT: motivations

Advantages of an external scheduler:

1. **Rapid experimentation**, avoiding the need to recompile the kernel and reboot the host;
2. **simple API** and **safety features**;
3. **Bespoke (/specific) schedulers** can be created;
   a) Meta could improve Facebook's web service RPS (Requests Per Second) by 1.25-3% and P99 (99th percentile) latency by 3-6%.
4. **Quick rollouts** of policies (i.e. temporary fixes);
5. Allows moving some decisions to **userspace**, using maps for userspace-kernelspace communication.



SCHED_EXT: counterarguments

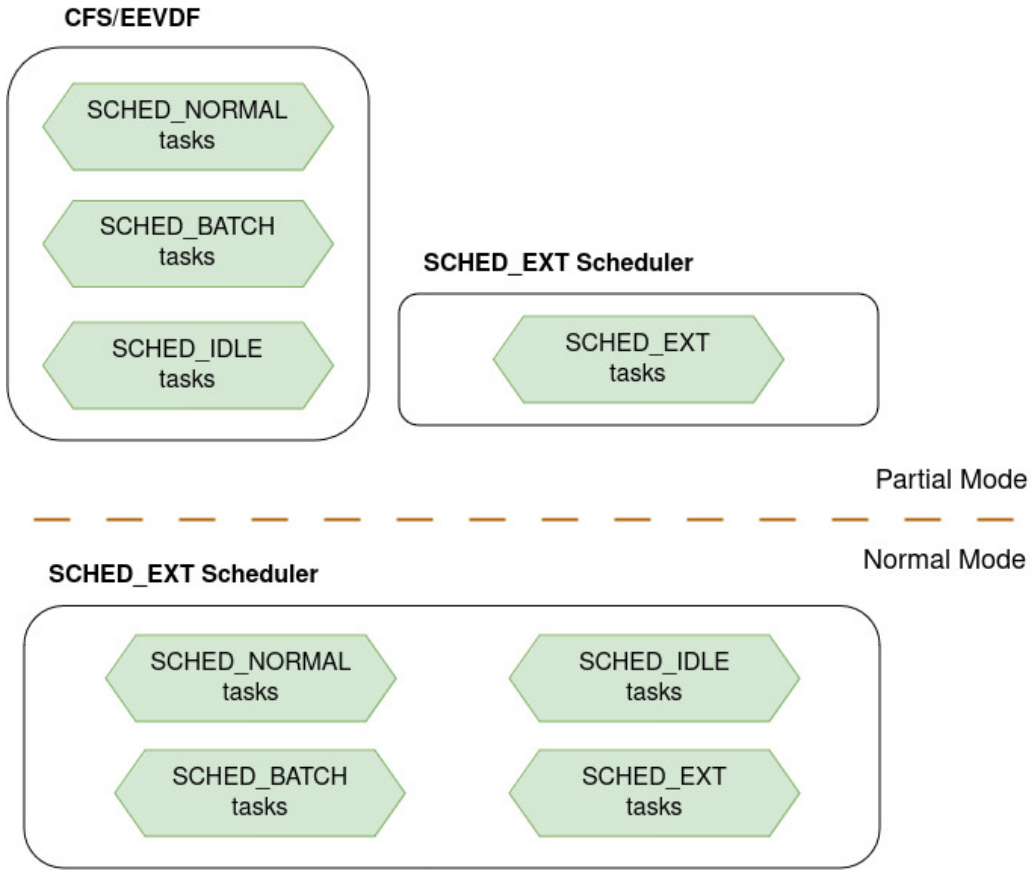
Even though using SCHED_EXT has numerous advantages, these are not enough to dispel all of the concerns.

1. **Inhibition of contributions** to the mainline scheduler;
   a) Creation of **out-of-tree non-GPL** schedulers.
2. **Program-specific schedulers** can be created and required by some programs (causing incompatibilities);
3. Wheter the **kernel should be tainted** (th kernel mark itself as such when somethir occurs that might be relevant later when investigating problems).

SCHED_EXT: integration in Linux

- In the kernel SCHED_EXT is implemented as a **new scheduling class** with priority lower than SCHED_NORMAL;
- SCHED_EXT has two different modes of operation:

1. **Partial mode**: tasks with SCHED_EXT priority are scheduled by the external scheduler while SCHED_NORMAL,… ones are scheduled by CFS/EEVDF;
2. **Normal mode**: in which all tasks with SCHED_NORMAL priority are set to be scheduled by the external scheduler.



SCHED_EXT: state of development

Currently, it requires **building the kernel** with the proposed patch and with certain **build flags**.

While its upstreaming in the kernel has been discussed for a long time, they have now **decided to merge it** for the 6.11 kernel release, scheduled for September of this year.

Linux Torvald announced:

"And using the 'in order to accept this, some other thing has to be fixed first' argument doesn't really work well either […]"

"I'm also not a believer in the argument that has been used (multiple times) that the BPF scheduler would keep people from participating in scheduler development. I personally think the main thing that keeps people from participating is too high barriers to participation."

Writing custom scheduler: sched_ext_ops

The first step to writing custom schedulers is the definition of a particular *struct* called sched_ext_ops. This contains:

1. **all callback implemented** in the custom scheduler provided as a pointer to a function;
2. **Flags** for changing default behaviour;
3. **Timeout:** the maximum amount of time a task can wait consecutively to run before the scheduler is unloaded;
4. **Name**.

```
SEC(".struct_ops.link")
struct sched_ext_ops rustland = {
    .select_cpu = (void *)rustland_select_cpu,
    .enqueue = (void *)rustland_enqueue,
    .dispatch = (void *)rustland_dispatch,
    .running = (void *)rustland_running,
    .stopping = (void *)rustland_stopping,
    .update_idle = (void *)rustland_update_idle,
    .set_cpumask = (void *)rustland_set_cpumask,
    .cpu_release = (void *)rustland_cpu_release,
    .init_task = (void *)rustland_init_task,
    .exit_task = (void *)rustland_exit_task,
    .init = (void *)rustland_init,
    .exit = (void *)rustland_exit,
    .flags = SCX_OPS_ENQ_LAST | SCX_OPS_KEEP_BUILTIN_IDLE,
    .timeout_ms = 5000, // 5 seconds
    .name = "rustland" // scheduler name
};
```

Example from Rustland scheduler

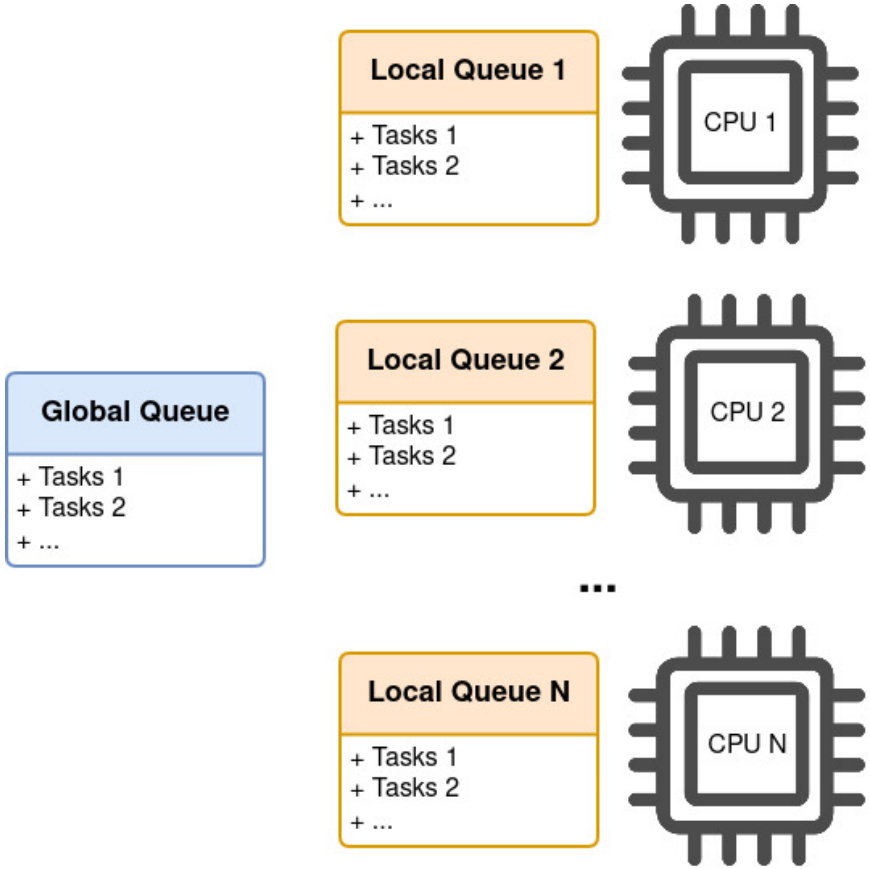
Writing custom scheduler: dispatch queue

Dispatch queues in SCHED_EXT are **queues of tasks** (either FIFO or priority queues). The dispatch queues are of three types:

1. **local queue**: one for each CPU, from which it executes tasks;
2. **global queue**;
3. any number of **custom queues**.

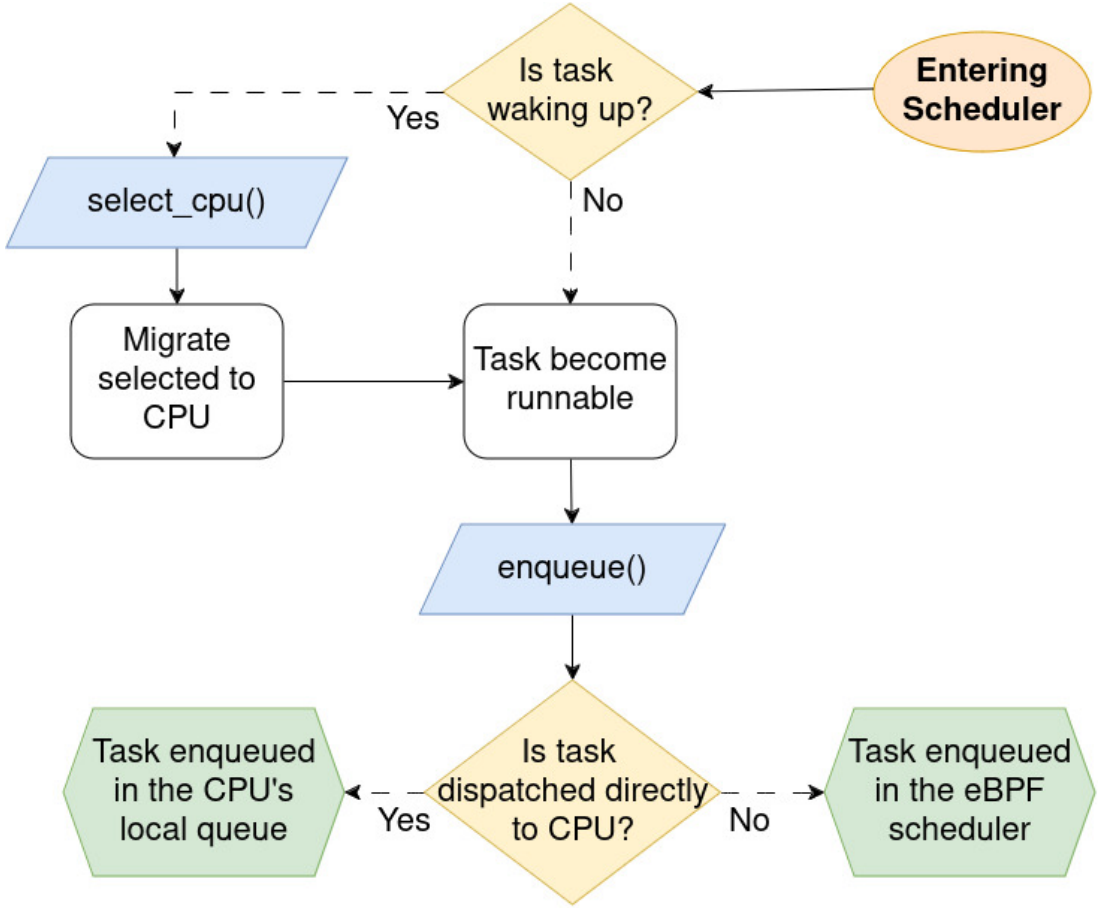
Each scheduling operation can be achieved by **dispatching**, moving tasks between queues.

- Moving to a local queue represents **running** the task on that CPU.
- Non-local queues can be **consumed**: its first task is moved to a local queue.



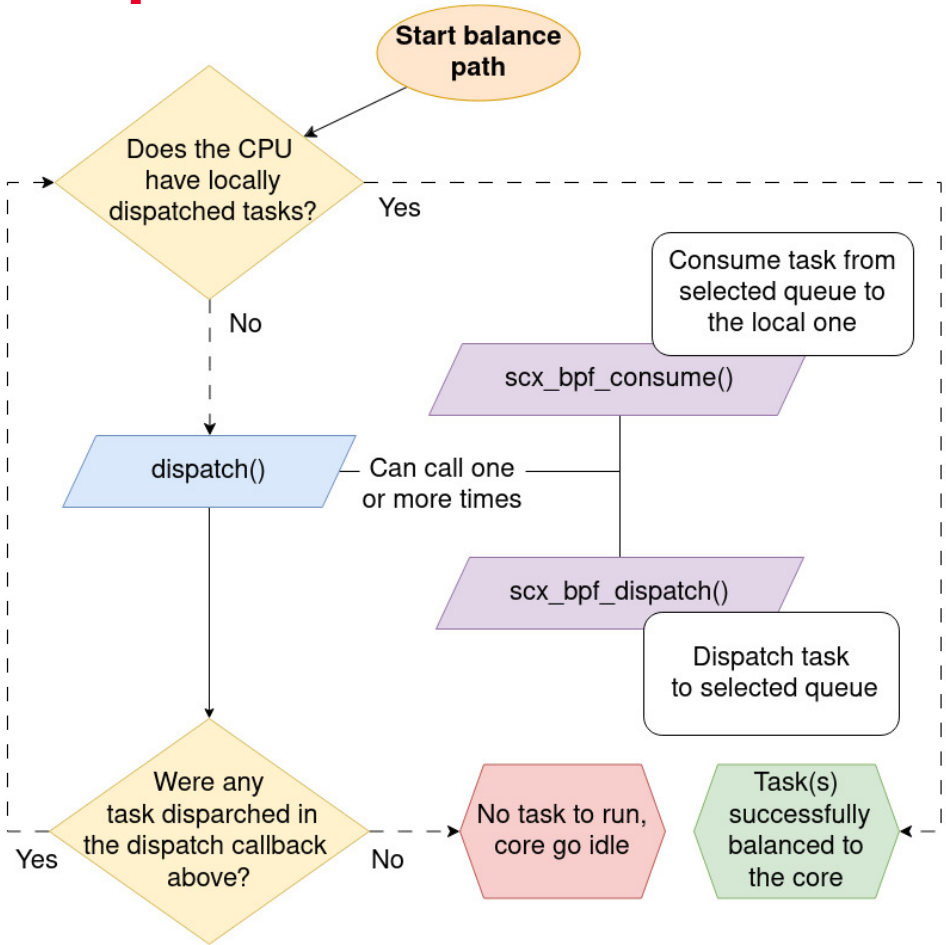
Writing custom scheduler: callbacks to implement

1. **prep_enable()**: used for initialization before enable();
2. **enable()**: enables the scheduling of new tasks;
3. **select_cpu()**: selects the CPU that will run the task, as an optimization;
4. **enqueue()**: enqueues a task into the scheduler so that it will be run.



Writing custom scheduler: callbacks to implement

5. **dispatch()**: called when a CPU queue is empty to assign a new task to keep the CPU busy;
6. **runnable(), running(), stopping(), quiescent()**: notifies of the new status of a task;
7. **update_idle()**: notifies of the new state of idleness of a CPU core;
8. **cpu_acquire(), cpu_release()**: notifies of a change in core availability.



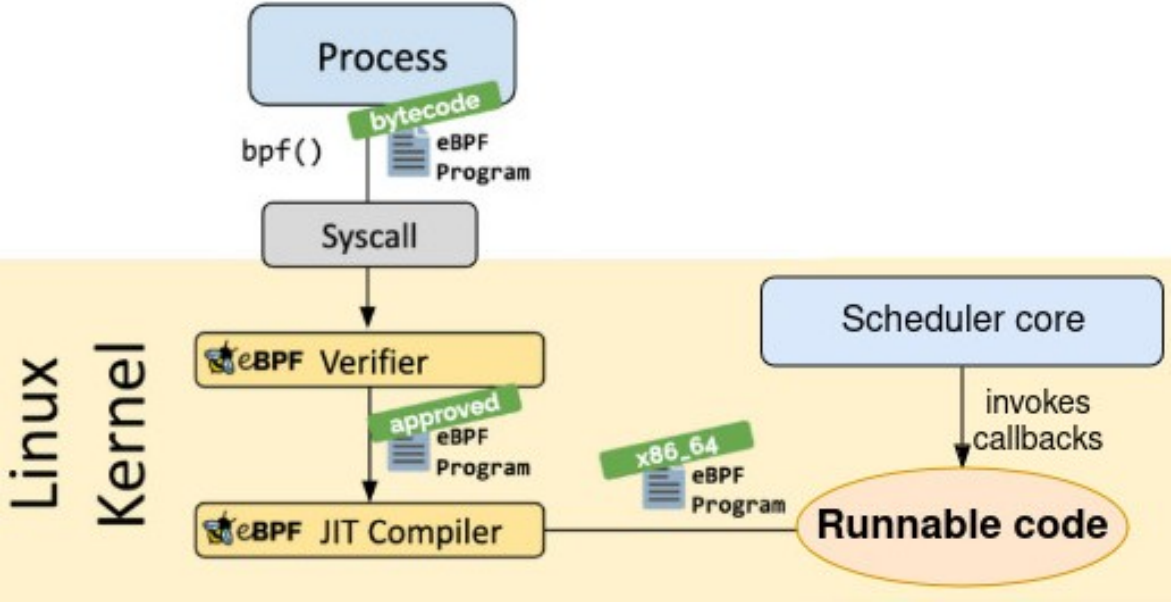
Loading a scheduler at run-time: eBPF

To load the scheduler two elements are required:

1. a **system call** to do so;
2. a **language** that supports this.

That language is eBPF, a safe JIT (Just In Time) that can be run in the kernel. Its features are:

- **Performance**: its ISA is aligned with x86 ISA allowing JIT compilation;
- **Safety**: eBPF code is statically analyzed before it is loaded into the kernel.



Conclusion

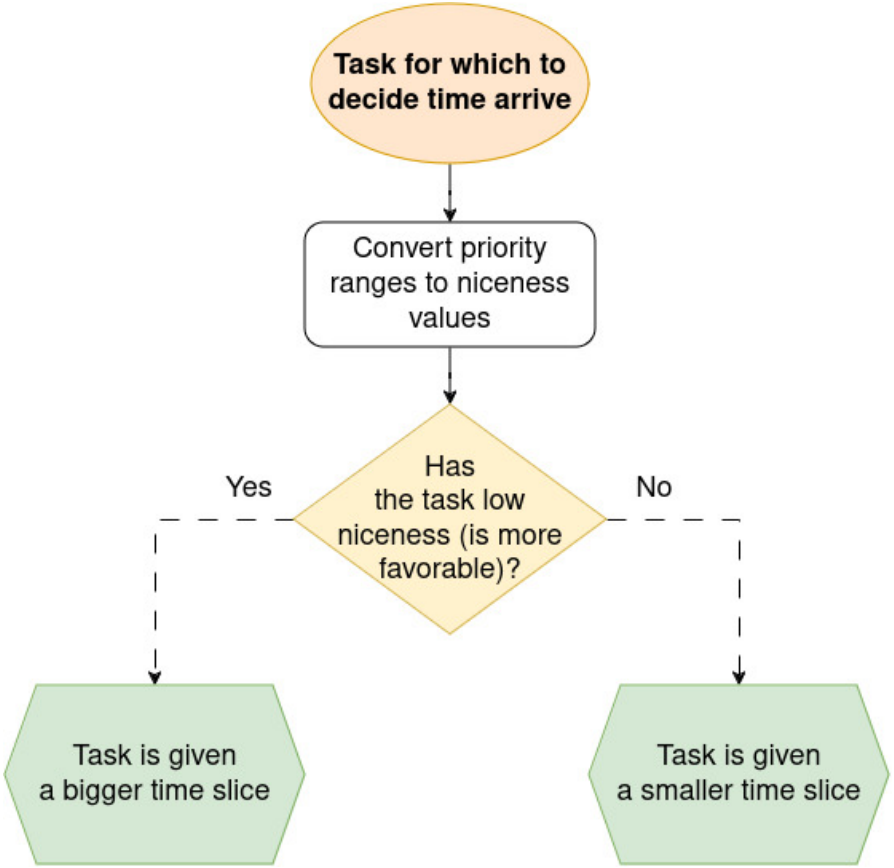
In conclusion, my research on the SCHED_EXT expanded my understanding and skills regarding scheduling mechanisms. The key insights and experiences gained are:

1. **Kernel Scheduling Knowledge**: I have gained a comprehensive understanding of Linux kernel task scheduling;
2. **eBPF Proficiency**: Developed skills in writing and deploying eBPF programs;
3. **SCHED_EXT Experience**: Hands-on experience in loading and managing external schedulers;
4. **Scheduler Code Analysis**: Analyzed internal code of standard Linux schedulers;
5. **Real-World Applications**: Recognized practical applications of dynamic scheduler loading.

This research not only has enhanced my technical expertise but it also has provided me with a further appreciation for the complexities of modern operating system schedulers and the capabilities of its maintainers.

Extra Slides

Simple scheduler example: scheduling idea




```c
static u64 get_time_slice(struct task_struct *task){
  // priority in standard goes with range [100, 139]
  // convert to niceness with range [-20, 19]
  // with -20 most favorable and 19 least favorable
  const int norm_prio = task->prio - 120;
  int msecs;

  if(norm_prio <= 0){
    stat_inc(3); /* increase "big queued" stats */
    msecs = 20;
  }else{
    stat_inc(2); /* increase "small queued" stats */
    msecs = 10;
  }

  return msecs * NSEC_PER_MSEC;
}
```

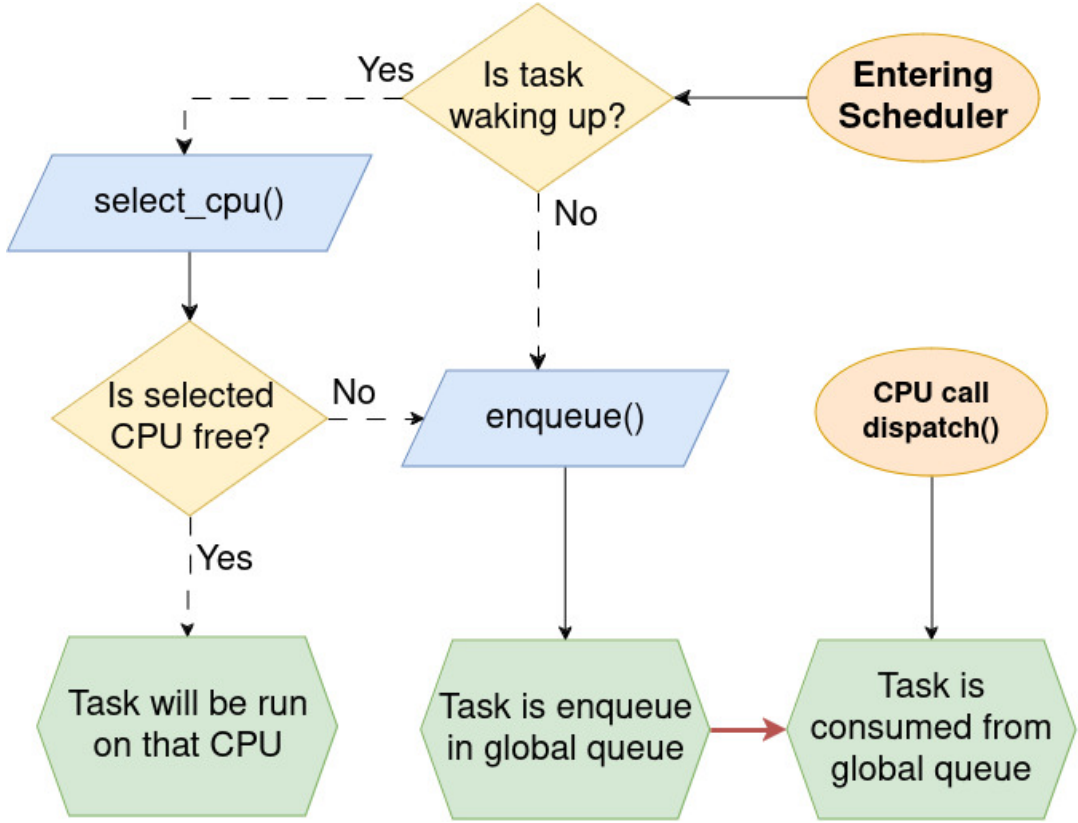
Simple scheduler example: scheduling functions

```
s32 BPF_STRUCT_OPS(simple_mine_select_cpu, struct task_struct
↪  *p, s32 prev_cpu, u64 wake_flags)
{
  bool is_idle = false;
  s32 cpu;
  cpu = scx_bpf_select_cpu_dfl(p, prev_cpu, wake_flags,
  ↪  &is_idle);
  if (is_idle) {
    stat_inc(0); /* increase "local queue" stats */
    scx_bpf_dispatch(p, SCX_DSQ_LOCAL, get_time_slice(p), 0);
  }

  return cpu;
}

void BPF_STRUCT_OPS(simple_mine_enqueue, struct task_struct *p,
↪  u64 enq_flags)
{
  stat_inc(1); /* increase "global queue" stats */
  scx_bpf_dispatch(p, SHARED_DSQ, get_time_slice(p),
  ↪  enq_flags);
}

void BPF_STRUCT_OPS(simple_mine_dispatch, s32 cpu, struct
↪  task_struct *prev)
{
  scx_bpf_consume(SHARED_DSQ);
}
```



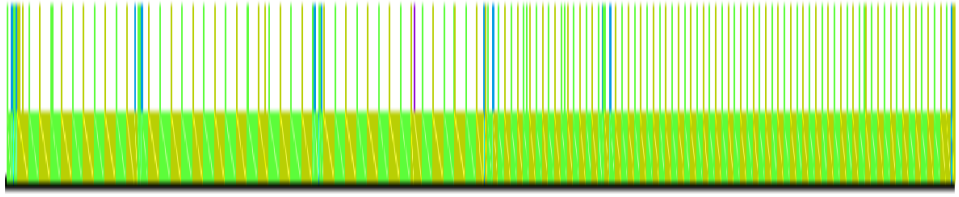
Simple scheduler example: workload and Kernelshark

Then, a **workload** is created that runs two processes which simply waste clock cycles on the CPU, having them share a single core (using a **CPU affinity mask**).

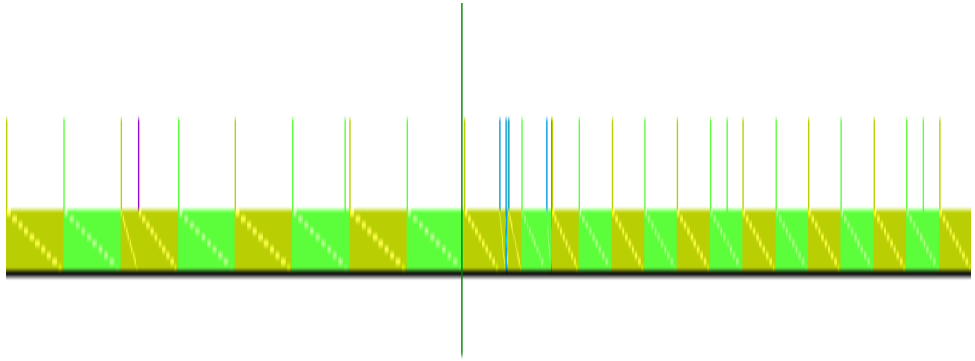
This workload also changes the priorities of the two processes during execution to demonstrate the time slice change because of the custom scheduler.

To show this change, two tools are used:

1. **trace-cmd** to get a trace of the execution of these two processes;
2. **KernelShark** to get a visual representation of the data by trace-cmd.



Graph given by KernelShark



Graph zoomed over the priority change